

VoxGuard: A Deterministic Policy-Execution Layer for Reliable Car Voice Assistants

Huang Chung-Yi

Independent Researcher — Team VoxGuard, CAR-bench Challenge 2026 (Track 1)

fongse@gmail.com

Abstract

We present VoxGuard, our Track 1 submission to the CAR-bench Challenge. VoxGuard inserts a deterministic policy-execution layer between the LLM and the tool interface, built from two components: a macro router that resolves recurring intents through hand-verified state machines without any LLM inference, and a guard chain that intercepts, validates, and rewrites the remaining LLM tool calls before they reach the environment. Using mid-tier GPT-5.4, VoxGuard reaches a task-type-average Pass¹ of 94.7% (123/129 tasks) on the public training split — measured end-to-end on the packaged container image, on the same split used for development — against the official 67% of the strongest frontier baseline (Claude Opus 4.6), at \$20.3 list-price-equivalent agent cost per full evaluation round, with 16% of tasks answered at zero LLM latency. We argue the baseline-to-ceiling gap on CAR-bench is an agent-architecture problem rather than a model-capability problem, and that determinism is the highest-leverage design axis under the official Pass^k metric, which penalizes cross-trial variance. All scaffolding is keyed exclusively to the public training split.

1 Introduction & Approach

CAR-bench [Kirmayr et al., 2026] evaluates LLM agents as car voice assistants across task completion (Base), hallucination detection (Hallucination), and ambiguity resolution (Disambiguation). Frontier models with minimal scaffolding reach 54–58% average Pass³ (official leaderboard); the strongest, Claude Opus 4.6, averages 67% on the weaker single-round Pass¹. Building toward the 90%+ range, we identified three root causes for the gap:

1. API semantic ignorance — models do not know environment-specific semantics (e.g., that `navigation_delete_destination` auto-promotes the previous waypoint), producing action sequences that hash outside the ground-truth state set.

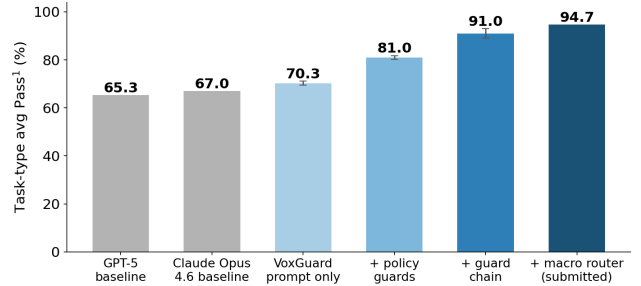


Figure 1: Task-type-average Pass¹. Baselines from the official results table; each VoxGuard stage adds deterministic components. Bars are single-round point estimates; whiskers show observed cross-round variance where repeated rounds were run. The submitted bar reflects the full final lineage (macro router plus subsequent gating and compliance hardening); the macro router’s specific contribution is variance, cost, and latency elimination (Figure 3), which compounds under Pass^k.

2. Policy blindness — documented policies (unit conventions, read-before-mutate prerequisites, disclosure obligations) are applied inconsistently across sampling runs.
3. Over-eagerness — when tools or parameters are removed (Hallucination split), models attempt workarounds instead of honestly refusing.

All three share one property: they are deterministic failure modes with deterministic fixes. Our approach is therefore to move every decision that can be resolved by documented policy or environment state out of the LLM and into code, reserving LLM inference for genuinely open-ended language understanding. This directly targets the official metric: Pass^k [Yao et al., 2024] requires passing all k trials, so every LLM judgment replaced by deterministic code removes a variance source at compounding value.

2 Model & Harness Architecture

The user message first hits the macro router; on a match, a deterministic state machine emits the full tool-call sequence with zero LLM inference. Unmatched messages

flow to the LLM (GPT-5.4 [OpenAI, 2026]), whose tool calls then pass through the guard chain, which can block, rewrite, inject context, re-prompt, or append disclosures before anything reaches the environment.

2.1 Macro Router: Planning–Execution Separation

The macro router matches the opening utterance against intent keyword specs and, on a hit, executes a hand-coded state machine: read state, ask clarifying questions when the task is ambiguous, then mutate. Two flavours exist: declarative MacroSpecs (keyword match + step list) for single-action intents, and hand-coded classes for conditional flows — e.g., weather-conditional navigation: resolve the city ID at runtime, compute the weather-query time as departure time plus fastest-route duration, and re-route on rain.

Three design rules keep macros sound and compliant:

- Values are derived at runtime, never memorized. Target temperatures and heating levels are parsed from the user’s reply; opening percentages and colors come from stored user preferences; city IDs from lookup tools. A macro that cannot derive a value returns control to the LLM.
- Safe fallback. Any unmatched context, missing field, or unparseable reply falls through to the LLM path — on a held-out task the worst case is the LLM baseline, never a wrong deterministic answer.
- Honest degradation. Macros check tool availability and parameter schemas before matching; on Hallucination variants they either refuse honestly or acknowledge missing readings while completing the unaffected main task.

In the submitted round, macros fully resolve 21/129 tasks (16%) with zero LLM tokens and 0.00s median LLM latency (vs. 34.9s median for LLM-path tasks), and these tasks are deterministic across trials by construction (Figure 3).

2.2 Guard Chain: Policy-Check Pass

Tasks that reach the LLM pass through ~ 15 guards (cf. programmable rails [Rebodea et al., 2023]) that see only benchmark-allowed inputs (system prompt, transcript, tool definitions, tool results). Representative guards:

- Capability Firewall: a contract table mapping each capability to its exact tool; if the required tool is missing and the LLM proposes a substitute from the contract’s substitution set, all calls are blocked and a refusal is emitted — converting the “should I try a workaround?” judgment into code.
- Policy DAG: read-before-mutate prerequisites (e.g., `set_air_conditioning` requires prior climate and window reads); violating calls are stripped or prerequisites injected.
- Weather-ETA Rewriter: for conditional navigation, rewrites `get_weather` timestamps to the arrival time

computed from tracked route durations, and enforces route-before-weather ordering.

- Window-Shape Guard: applies the documented $>20\%$ window-closing threshold — close per-window when a small opening must be preserved, ALL otherwise — derived from policy documentation and verified against training tasks.
- Obligation Renderer / Promise Check: appends required disclosures (route rationale, alternatives offer) and re-prompts when the agent’s text promises an action its tools cannot deliver.

2.3 Verification Pyramid

Guard and macro logic is enforced by a four-level test stack: L0 unit tests (118) on macro/guard logic; L1 a routing snapshot replaying 460 paraphrased opening messages from all 129 training tasks against the matcher, diffing the routing table on every keyword change (this caught six mis-grab regressions that single-run scores had disguised as “LLM variance”); L2 single-task end-to-end runs for directly affected tasks; L3 full-split rounds. L0/L1 run in seconds, enabling 30+ code changes in the final week with zero uncaught routing regressions.

3 Design Decisions

Compliance first. The rules prohibit encoding test-set answers while explicitly permitting agentic scaffolding, lookup tables without test answers, and fine-tuning on the provided training data. We adopted a stricter internal line: no macro may emit a literal value whose only provenance is a training ground truth — every emitted value must trace to dialogue, stored preferences, tool results, or documented policy. Beyond compliance, this rule is what makes macros transfer: a parsed value generalizes to held-out tasks while a memorized one does not. A release audit verified: zero test-split task references in code, zero runtime access to task definitions or answer keys, and all guard rules re-derivable from policy documentation plus training tasks alone.

Determinism as the Pass^k strategy. Cross-run analysis showed single-round scores mask ± 3 –5 tasks of sampling variance. We therefore classified every failure as stable (fails every round $\rightarrow$ fix with a deterministic guard or macro) or variance (fails some rounds $\rightarrow$ architectural mitigation only), and spent engineering effort exclusively on the former. Each macro-absorbed task is variance-free by construction, which compounds under Pass^k.

The commit barrier. State-changing calls are never issued until all decisions are resolved: don’t commit without the required tool (Capability Firewall), before prerequisites (Policy DAG), before the user chooses among options (route-ask gating), or before knowing the ETA (Weather-ETA Rewriter). After committing, disclose (Obligation Renderer).

4 Validation Results

The submitted build scores 123/129 tasks single-round (Table 1), measured end-to-end on the pack-

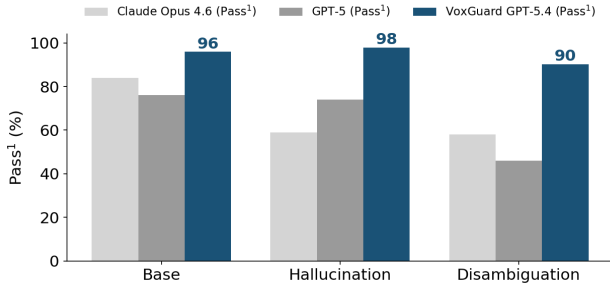


Figure 2: Per-split Pass¹ vs. official frontier baselines. The largest gains are on Disambiguation (+32.3 over the best baseline) and Hallucination (+23.9), the two splits dominated by deterministic policy application.

Configuration	Base	Hall	Dis	Avg
Opus 4.6 baseline	84	59	58	67.0
GPT-5 baseline	76	74	46	65.3
VoxGuard, prompt only	78	81	52	70.3
+ policy guards	80	92	71	81.0
+ guard chain	92	93.8	87.1	91.0
+ macro router (submitted)	96	97.9	90.3	94.7

Table 1: Single-round Pass¹ (%) by task type; Avg is un-weighted across task types, matching the official leaderboard convention. Baseline rows are the official Pass¹ results; VoxGuard rows are our measurements on the public training split, each a development-lineage snapshot. The submitted row is the final build lineage, measured end-to-end on the packaged container image.

aged container image (the submitted image adds only a linux/amd64 rebuild and an env-handling fix, both smoke-validated on the submitted digest). Predecessor submission-candidate builds measured 115–121/129 across full rounds in the final development days. Per-task attribution of the submitted round’s 6 failures: 4 are stable known-hard tasks (multi-leg navigation with charging constraints; route-choice judging; a window-semantics disambiguation) and 2 are recurring round-dependent flakes whose transcripts show sampling-dependent simulator or judge behavior rather than logic errors. All 21 macro-resolved tasks passed with zero LLM inference; a macro-phase gap observed in a predecessor round (an unguarded LLM turn between macro phases) was closed by a phase-gating fix, regression-tested red-to-green before packaging, and the affected task passes in the submitted round.

Failure taxonomy. Across the submitted round and predecessor full rounds, the failures stable across rounds concentrate in route-choice judging (the judge expects proactive fastest-route selection under some phrasings and a clarifying question under others) plus isolated known-hard tasks (multi-leg charging, a window-semantics reading); the remaining failures alternated across rounds — confirming variance, not logic, as the residual bottleneck.

Guard	Attributed impact
Capability Firewall	−2 tasks (stable)
Celsius/unit guard	−3 to −5 tasks
Unknown-field detection	−2 to −4 tasks
Lighting/fan Policy DAG	−2 tasks
Obligation Renderer	−1 task
Weather-ETA Rewriter	prevents cond.-nav ordering fails

Table 2: Per-guard impact attribution, estimated from guard trigger counts and development-round score deltas; guards were not individually disabled for full ablation re-runs.

5 Guard Attribution & Lessons Learned

Lessons. (1) Read-only $\neq$ score-safe: one failed lookup call zeroes `r_tool_execution` for the whole task; tool risk is a property of metric impact, not state mutation. (2) Snapshot the router, not just the score: six “LLM variance” failures were actually macro mis-grabs on emotional keywords (“stuff”), found only when L1 replayed paraphrases against the matcher. (3) String-matching bugs hide in plain sight: single-quote repr vs. double-quote JSON, and word-boundary regexes against `snake_case`, silently disabled two guards for several versions — behavioral unit tests, not eyeballing, caught them. (4) Confirmation texts are policy surface: the LLM judge requires tool parameters restated in confirmation questions; a one-line template fix flipped two tasks.

Reliability lives in the architecture, not the model. Our central claim — that the baseline-to-ceiling gap is architectural — makes a falsifiable prediction: if policy compliance, refusal behavior, and confirmation flows are enforced in code, then swapping the LLM for a weaker tier should cost nothing wherever the architecture carries the load, and should hurt only where open-ended planning remains with the model. We tested this prediction from both ends of the difficulty spectrum (Table 3), and it held on both: on simple intents GPT-5.4-mini matches the full model (the single divergence is a documented cross-round flake, and all ten Hallucination tasks pass on mini precisely because refusal is code, not model); on the hard tail, mini-specific failures concentrate in route-choice and disclosure-heavy dialogue, delimiting the segment that must stay on the full tier; macro-covered tasks are model-independent by construction. Routing intents by these measured segments projects a $\sim 50\%$ cost reduction at score parity ($\$20.3 \rightarrow \sim \10.2 per round). Caveats: probe subsets are small (30/14 tasks; mini figures are from a single full round, while the full-model route-choice range spans repeated rounds); probes were measured on the immediate predecessor build (the submitted build’s phase-gating fix does not alter LLM-path routing, so segment boundaries should transfer, but they were not re-measured); the 50% figure is a projection rather than an end-to-end measurement; and mini’s median latency is slightly higher (more tokens at a lower unit price). A learned risk router is left to future work.

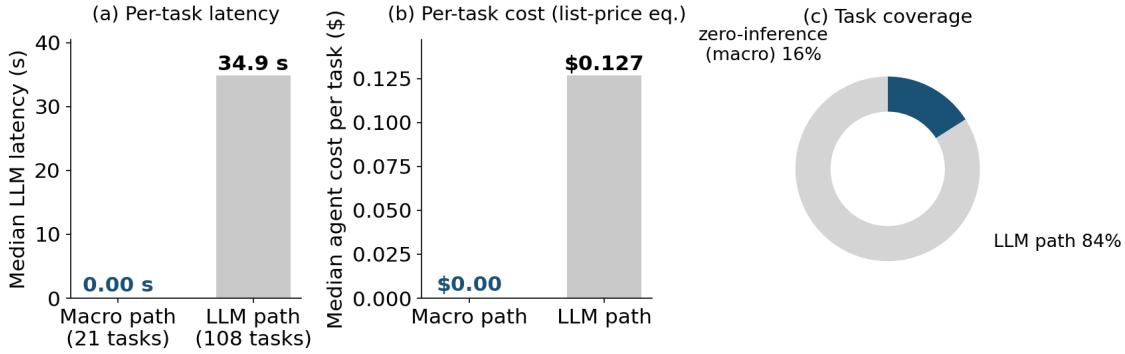


Figure 3: Efficiency profile of the submitted build: (a) macro-path tasks incur zero LLM latency; (b) per-task list-price-equivalent agent cost by path; (c) 16% of tasks complete with zero LLM inference.

Segment	GPT-5.4	5.4-mini	Cost	Conclusion
Simple intents (30 tasks)	30/30	29/30	−67%	VoxGuard shows that on CAR-bench the Pass ¹ distance between the strongest frontier baseline’s 67% and ~95% is closed not by a larger model but by a deterministic policy-execution layer: macros for the head of the intent distribution, guards for the tail, and a verification pyramid that keeps both honest. Determinism is not only a reliability strategy — under Pass ^k it is the scoring-optimal one, and it comes with lower cost and latency than frontier-model brute force.
Hard tail: route choice (14)	6–10/14	4/14	−75%	
Macro-covered (22 on probe build)	model-indep.		−100%	

Table 3: Two-sided tier probes. Mini matches the full model on simple intents and fails specifically on route-choice reasoning — the measured boundary for tiered routing.

6 Compute & Rate-Limit Assumptions

Agent LLM: GPT-5.4 (high reasoning effort) behind an OpenAI-compatible local proxy; user simulator and policy judge per benchmark defaults. Measured on the submitted round (full 129 tasks, packaged container image): \$20.3 total agent-side cost — list-price equivalent computed by the benchmark harness (LiteLLM [BerriAI, 2025] per-call response cost from reported token usage at GPT-5.4 API list prices; median \$0.11/task) — median 29.3s end-to-end LLM latency per task (p90 129s), median 4 A2A [A2A Project, 2025] turns; 16% of tasks complete with zero LLM calls. No fine-tuning and no additional GPUs; the system is prompt + Python middleware, model-agnostic by construction. Development consumed a single API quota; we assume the official evaluation permits comparable per-task latency.

7 Compliance Statement

All VoxGuard components are keyed exclusively to the public 129-task training split; we never accessed, probed, or inferred hidden test-set content. Macros map training-task intents to procedures whose values are computed at runtime from tool results, stored preferences, and dialogue — permitted as agentic scaffolding and pre-computed lookup tables without test-set answers, analogous to the explicitly permitted fine-tuning on provided training data. No component reads answer keys, task definitions, or evaluator internals at runtime; auxiliary passes see only the system prompt, transcript, tool definitions, and tool results.

References

- [A2A Project, 2025] A2A Project. A2A: Agent2Agent protocol. <https://github.com/a2aproject/A2A>, 2025.
- [BerriAI, 2025] BerriAI. LiteLLM: A unified interface for LLM APIs. <https://github.com/BerriAI/litellm>, 2025.
- [Kirmayr et al., 2026] Johannes Kirmayr, Lukas Stapfen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [OpenAI, 2026] OpenAI. GPT-5.4 model documentation. <https://developers.openai.com/api/docs/models/gpt-5.4>, 2026.
- [Rebedea et al., 2023] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails, 2023.
- [Yao et al., 2024] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.